

Introduction

Independent component analysis (ICA) is the canonical approach for recovering statistically independent latent signals from observed linear mixtures. The classic application is the “cocktail-party problem” — separating individual conversations from microphone recordings of overlapping speakers — but the technique generalises to any setting where multiple independent sources mix linearly into a smaller set of observed channels. EEG, fMRI, gene-expression data, and audio source separation all use ICA as a primary decomposition method.

Prerequisites

A working understanding of principal components analysis, statistical independence (a stronger condition than uncorrelatedness), and the concept of a linear mixing model.

Theory

The ICA model assumes observations X are generated as $X = AS$, where A is an unknown $n \times n$ mixing matrix and S contains the independent latent sources. The goal is to estimate A (or its inverse, the unmixing matrix W) so that $\hat{S} = WX$ recovers the sources up to permutation and scale ambiguities.

Two principles drive ICA algorithms:

- **Non-Gaussianity maximisation:** by the central limit theorem, sums of independent variables become more Gaussian; an unmixing direction that maximises non-Gaussianity (kurtosis, negentropy) recovers an independent source.
- **Statistical independence maximisation:** minimise mutual information between recovered components.

FastICA is the most widely used algorithm and uses fixed-point iteration on kurtosis or negentropy. Infomax is the alternative principle implemented in domain-specific tools.

The key contrast with PCA: PCA produces *uncorrelated* components by maximising variance; ICA produces *independent* components by maximising non-Gaussianity. The two coincide only for Gaussian data, where ICA is undefined.

Assumptions

Sources are statistically independent and non-Gaussian (at most one Gaussian source can be recovered); mixing is linear and instantaneous; the number of sources equals the number of observed channels (or equals the chosen number of components).

R Implementation

```
library(fastICA)

set.seed(2026)
# Simulate two sources: sine and sawtooth
t <- seq(0, 8, length.out = 1000)
s1 <- sin(2 * pi * t)
s2 <- ((2 * t) %% 2) - 1      # sawtooth

# Mix
A <- matrix(c(1, 0.5, 0.5, 1), 2, 2)
S <- cbind(s1, s2)
X <- S %*% t(A)

# Run ICA
```

```
ic <- fastICA(X, n.comp = 2)

par(mfrow = c(3, 1))
matplot(S, type = "l", main = "True sources")
matplot(X, type = "l", main = "Mixed signals")
matplot(ic$S, type = "l", main = "Recovered ICA components")
par(mfrow = c(1, 1))
```

Output & Results

`fastICA()` returns the recovered sources `$S`, the estimated mixing matrix `$A`, and the unmixing matrix `$W`. The three-panel plot shows the original sine and sawtooth sources, the mixed signals as observed, and the ICA-recovered components — visually almost identical to the originals up to sign and permutation.

Interpretation

A reporting sentence: “FastICA on two channels of mixed sine and sawtooth signals recovered both sources cleanly (visual inspection), demonstrating blind source separation under the linear-mixing model; recovered components were rescaled to unit variance and permuted, reflecting the inherent ICA ambiguities.” Mention the algorithm and pre-processing (whitening, mean-centring) in the methods section.

Practical Tips

- Pre-whiten the data (PCA-based decorrelation followed by unit-variance scaling) before FastICA; the `fastICA` package does this automatically.
- ICA is essentially a rotation of whitened data — order and sign of components are arbitrary; do not interpret component indices as meaningful.
- At most one Gaussian source can be recovered; if all sources are Gaussian, ICA cannot separate them.
- For fMRI and EEG, specialised implementations (MELODIC in FSL, runica in EEGLAB) handle the domain-specific challenges of large channel counts and physiological artefacts.
- For non-square mixing problems (more sources than channels), compressive ICA and multi-modal extensions exist but require specialised tools.
- ICA is unsupervised; evaluating the result requires interpreting the recovered signals substantively, often through expert judgement.

R Packages Used

`fastICA` for the standard FastICA implementation; `ica` for additional ICA variants including extended Infomax; `mlbench` for synthetic mixing examples; `JADE` for joint approximate diagonalisation of eigen-matrices, an alternative algebraic approach to ICA.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- PCA, factor analysis, CCA, LDA
- Clustering: k-means, hierarchical, model-based

- UMAP and t-SNE